Roteiro: `peer/host.py` (UDP)

Passo a passo para implementar o **host** da partida. Cada passo termina com um **teste** para você saber que pode seguir. Não pule o teste: um bug no passo 4 costuma só aparecer no passo 12.

Leia antes: `README.md` seções 8 (algoritmo do ciclo), 10 (mensagens UDP), 11 (estados), 12 (mensagens inválidas), 15 (falhas), e `docs/divisao-tarefas.md` seção 2 (contratos).

Nunca use `print` com o `curses` aberto. Use `logging` para `logs/host.log` e acompanhe em outro terminal com `tail -f logs/host.log`.

O que o host faz (visão de cima)

```
COLLECTING ──► COUNTDOWN ──► RUNNING ──► ENDING ──► FINISHED
     │                          │
     └── < 2 jogadores ──► ABORTED ◄── Q (saída do host)
```

Estado	O que acontece
`COLLECTING`	Recebe `HELLO`, responde `WELCOME`. Prazo: `hello_deadline_ms`
`COUNTDOWN`	Manda `TICK` vazio (sinal de vida) durante `countdown_ms`
`RUNNING`	A cada `tick_ms`: aplica `step`, guarda o registro, manda `TICK` a cada peer
`ENDING`	Continua mandando o registro final até todos confirmarem ou dar timeout

---

Passo 0: preparar

- ☐ Criar o branch: `git checkout -b feat/host`.
- ☐ Ler as seções do README listadas acima. Anote dúvidas.
- ☐ Combinar com C a assinatura do `step` e com E a do `protocol` (stubs no `dev`).
- ☐ Criar `peer/host.py` com a classe `Host` vazia e o método `run()`.
- ☐ Criar `logs/` e configurar o `logging` para arquivo.

Teste: `python3 -c "from peer.host import Host"` não dá erro.

Passo 1: o socket UDP

Conceitos a estudar: `socket.AF_INET`, `SOCK_DGRAM`, `bind`, `recvfrom`, `sendto`, `setblocking(False)`.

- ☐ No `__init__`, criar o socket e fazer `bind((bind_host, porta_udp_do_host))`. Porta e host vêm do `config` e do `match_start`, **não fixos**.
- ☐ Deixar o socket **não bloqueante**.
- ☐ Ler datagramas em um laço que **esvazia tudo o que chegou** até dar `BlockingIOError`.

Teste: use `nc -u 127.0.0.1 6001` em outro terminal, digite algo e veja o host registrar no log o conteúdo e o endereço de origem.

Passo 2: estrutura de dados dos jogadores

A partir de `match_start["players"]`, monte por `pid`:

```
jog[pid] = {
    "addr": (host, porta),       # de onde esse pid pode falar
    "iseq": 0,                   # maior iseq aceito
    "dir": None,                 # última direção desejada válida
    "ack": 0,                    # último ciclo confirmado pelo peer
    "visto": None,               # instante do último datagrama válido
    "conectado": True,
```

```
}
```

- ☐ O `pid` do host é o `host_slot` (0) e não tem `addr` de peer.
- ☐ Guardar um dicionário de endereço → `pid` para validar a origem rápido.

Teste: imprimir no log a tabela montada a partir de um `MATCH_START` de exemplo (copie o do README).

Passo 3: receber e validar um datagrama

Função `_tratar_datagrama(dados, addr)` que **descarta em silêncio** (só log) se:

- ☐ `decode` devolver `None`;
- ☐ `m` não for o `match_id`;
- ☐ `addr` não for o endereço registrado para o `pid` informado;
- ☐ a mensagem não fizer sentido no estado atual.

Só depois decide o que fazer, por tipo (`HELLO`, `INPUT`).

Teste: mande de outra porta um `HELLO` com o `pid` de outro jogador e confirme que ele é descartado.

Passo 4: `COLLECTING`, o handshake

- ☐ `HELLO` válido → marcar o jogador como presente e responder `WELCOME` com `welcome_state(estado, tick_ms)` e `countdown_ms`. **Todo** `HELLO` **recebido recebe um** `WELCOME`, mesmo repetido (o `WELCOME` pode ter se perdido).
- ☐ Sair de `COLLECTING` quando todos mandarem `HELLO` **ou** acabar `hello_deadline_ms`.
- ☐ Quem não mandou `HELLO` vai para a lista `desconectados` do primeiro `step` (motivo `DISC`).
- ☐ Se sobrarem menos de 2 jogadores, ir para `ABORTED` e mandar `ABORT`.

Teste: suba o host e um peer de teste (script de 15 linhas do `emissor.py` adaptado). O peer recebe `WELCOME`. Suba o peer **antes** do host e veja os reenvios do `HELLO` funcionando. Mande o `HELLO` duas vezes e confira dois `WELCOME`.

Passo 5: `COUNTDOWN`

- ☐ Durante `countdown_ms`, mandar a cada `tick_ms` um `TICK` com `recs: []` para todos os peers presentes.
- ☐ Acabado o tempo, ir para `RUNNING` e iniciar a numeração do ciclo em 1.

Teste: o peer de teste recebe `TICK` vazios a cada 100 ms durante 3 s e depois o primeiro com registro.

Passo 6: o laço de tempo fixo

Esqueleto (ideia, escreva com suas palavras):

```
proximo = time.monotonic()
enquanto rodando:
    esvaziar o socket (receber tudo que chegou)
    ler a tecla do host
    se time.monotonic() >= proximo:
        executar um ciclo
        proximo += tick_s            # soma, não "agora + tick_s"
    dormir só o que falta (no máximo alguns ms)
```

Cuidados:

- ☐ Use `time.monotonic()`, não `time.time()` (relógio de parede pode saltar).
- ☐ `proximo += tick_s` evita acumular atraso. Se ficou **muito** atrasado (por exemplo, mais de 5 ciclos), reajuste `proximo` para o presente em vez de rodar vários ciclos seguidos.
- ☐ Não use `time.sleep(tick_s)` puro: você perderia datagramas que chegam durante o sono.

Teste: conte no log o intervalo entre ciclos por 30 s. Deve ficar perto de 100 ms, sem tendência de crescer.

Passo 7: receber `INPUT`

- ☐ Descartar se `iseq` ≤ `jog[pid]["iseq"]` (duplicado ou atrasado).
- ☐ Aceitar: atualizar `iseq`, `dir`, `visto` e `ack` (**só se o novo `ack` for maior**).
- ☐ Validar `dir` ∈ {`U`,`D`,`L`,`R`} (o `decode` já filtra, mas confira).
- ☐ `INPUT` de peer eliminado: aproveitar só `ack` e `visto`.

Teste: mande `INPUT` com `iseq` 5, depois 3. O segundo é descartado.

Passo 8: o ciclo com `step`

- ☐ Montar `desejadas` com a `dir` de cada peer que mandou `INPUT` desde o último ciclo (mais a do host).
- ☐ Montar `desconectados` (passo 10).
- ☐ Chamar `registro = step(estado, desejadas, desconectados, n)`.
- ☐ Anexar `registro` ao `historico` (lista; `historico[n-1]` é o ciclo `n`).
- ☐ `n += 1`.
- ☐ Se `registro["end"]` não for `None`, ir para `ENDING` e guardar `n_final`.

Enquanto C não entregar o `step` real, use um **stub** seu que só move todos para a direita, para destravar o resto.

Teste: com o stub, os registros crescem a cada 100 ms e aparecem no log.

Passo 9: montar e enviar o `TICK`

Para cada peer presente:

- ☐ Pegar `historico[ack : ack+8]` (ciclos `ack+1` até `ack+8`, no máximo o último existente).
- ☐ Montar `{"v":1,"t":"TICK","m":match_id,"recs":[...]}` com o `protocol`.
- ☐ Conferir que o datagrama tem **no máximo 1200 bytes**. Se passar, reduza o número de registros.
- ☐ `sendto` para o endereço do peer. Mande **todo ciclo**, mesmo com `recs` vazio (é o sinal de vida do host).

Teste: no peer de teste, imprima os `n` recebidos. Descarte 1 de cada 3 `TICK` de propósito e veja os `n` perdidos chegarem no `TICK` seguinte.

Passo 10: timeouts de peer

- ☐ A cada ciclo, para cada peer conectado: se `agora - visto > peer_timeout_ms`, marcar `conectado = False` e incluir o `pid` em `desconectados` **uma vez só**.
- ☐ Depois disso, parar de enviar `TICK` para ele.
- ☐ Peer que nunca mandou `HELLO` já entrou como desconectado no passo 4.

Teste: `kill -9` no processo de um peer. Em ~3 s aparece `DISC` no registro e a partida continua para os outros.

Passo 11: o fim da partida (`ENDING`)

- ☐ Continuar enviando aos peers vivos os registros pendentes até todos terem `ack ≥ n_final`.
- ☐ Limite de espera: `peer_timeout_ms`.
- ☐ Depois, `FINISHED`: fechar o socket e devolver `{"result": ..., "winner": ...}` do `run()`.

Teste: perca de propósito o `TICK` que carrega o `end` e veja o fim chegar ao peer mesmo assim.

Passo 12: o host também joga

- ☐ Ler `ui.poll_input()` no laço do passo 6. Direção → `desejadas[0]`. `QUIT` → sair (passo 13).
- ☐ Manter um `GameView` no host e aplicar cada registro **local** com `view.apply_record(registro)`.
- ☐ Chamar `ui.draw(view)` a cada ciclo.
- ☐ Nomes para o placar vêm de `match_start["players"]`.

Teste: o host joga e vê a mesma tela que os peers, no mesmo ciclo.

Passo 13: saída do host e `ABORT`

- ☐ `QUIT` do host: mandar `ABORT` 3 vezes (melhor esforço) para cada peer, fechar o socket e devolver `{"result": "ABORT"}`.
- ☐ Capturar `KeyboardInterrupt` para o terminal não ficar quebrado (o `with TerminalUI()` já restaura a tela).
- ☐ Menos de 2 jogadores restantes em qualquer momento: conferir com o README se vale `ABORT` ou vitória e seguir o que está escrito.

Teste: feche o host e veja os peers encerrarem de forma controlada em até `host_timeout_ms`.

Passo 14: armadilhas conhecidas

- No Windows, `recvfrom` em UDP pode levantar `ConnectionResetError` quando um peer fecha. Trate e ignore.
- `sendto` com endereço errado levanta erro: confira o `addr` vindo do `MATCH_START` (`host` é string, `port` é int).
- Um `dict` com chaves inteiras vira chaves string no JSON. Se o `protocol` serializar `pid` como chave, lembre de converter de volta.
- Não bloqueie o laço em nenhum ponto (nem `input()`, nem `recv` de socket bloqueante).
- Registre no log **cada** descarte com o motivo. É assim que você acha bugs depois.

Passo 15: ferramentas de teste

Com a `simnet` de C:

- ☐ `--sim-loss 0.2`: a partida precisa terminar igual em todos os processos.
- ☐ `--sim-delay-ms 80`: o jogo fica mais lento, mas coerente.
- ☐ Um peer lento (atrasa 1 s): recebe vários registros de uma vez e se atualiza.
- ☐ Mensagens lixo com `nc -u`: nada trava, o log mostra descartes.

Passo 16: integração

1. Com **B (cliente)**: dois processos reais, sem lobby, portas fixas. Resolver incompatibilidades de campos e tipos.
2. Com **E (`main.py`)**: o `MATCH_START` real chega do lobby e o `Host(config, match_start).run()` roda direto.
3. Os três cenários obrigatórios do trabalho: partida completa, queda de jogador, queda do lobby durante a partida.

Checklist final do host

- ☐ Handshake com `WELCOME` idempotente.
- ☐ Ciclo fixo sem derivar.
- ☐ `TICK` com janela de `ack` e ≤ 1200 bytes.
- ☐ Origem e `iseq` validados; descartes registrados.
- ☐ `DISC` por timeout, uma vez só.

- ☐ Fim confiável.
- ☐ `ABORT` ao sair.
- ☐ Funciona com 20% de perda e entre máquinas diferentes.
- ☐ Você consegue explicar, sem olhar o código, o que acontece quando: um `TICK` se perde, um peer cai, o host cai, o lobby cai.

Para estudar (você precisa explicar na banca)

- Por que usar `monotonic` e `proximo += tick_s`?
- Por que o host responde `WELCOME` a cada `HELLO`?
- O que garante que todos os jogadores chegam ao mesmo resultado?
- Por que a janela de 8 registros e o limite de 1200 bytes?
- O que acontece se o `INPUT` de um peer demorar mais que um ciclo?
- Qual a desvantagem de o host ter latência zero?